

CodeAlpha Cyber Security Internship

TASK 3 — SECURE CODING REVIEW

Python Flask Login Application — Security Audit & Remediation Report

Scope note: This is a demonstration secure-coding audit of a small sample Python/Flask application created for the internship task. It is not an audit of CodeAlpha or any third-party production system.

1. Objective

The objective is to review application code for common security weaknesses, demonstrate how the weaknesses can occur, recommend secure-coding practices, and document remediation steps. This directly addresses the CodeAlpha Task 3 requirements to select an application, perform a code review, use manual/static-analysis methods, recommend best practices, and document findings and remediation.

2. Application Selected

Application: Python Flask login application

Technology: Python, Flask, SQLite

Functions reviewed: user registration, login, database access, session handling, and profile display.

3. Review Method

The review uses manual source-code inspection and a security checklist. The following areas were considered: input validation, database queries, password storage, session management, output encoding, secrets/configuration, error handling, and dependency/security hygiene.

4. Vulnerability Findings

ID	Finding	Risk	Severity
SC-01	SQL injection from string-built SQL queries	Unauthorized data access/manipulation	High
SC-02	Passwords stored as plaintext	Credential disclosure	Critical
SC-03	Hard-coded secret key	Session compromise if exposed	High
SC-04	Missing input validation	Malformed/unexpected input	Medium
SC-05	Unsafe HTML output	Possible XSS	High
SC-06	Overly detailed error messages	Information disclosure	Low/Medium

SC-01 — SQL Injection

Vulnerable pattern: SQL is constructed by concatenating user input into a query. Example: `query = "SELECT * FROM users WHERE username='" + username + "'"`.

Impact: An attacker may alter the intended SQL statement and potentially read or modify database data.

Remediation: Use parameterized queries/prepared statements. Never concatenate untrusted input into SQL.

SC-02 — Plaintext Password Storage

Vulnerable pattern: storing the submitted password directly in the database.

Impact: If the database is exposed, every stored password is immediately readable and may be reused on other services.

Remediation: Store passwords using a modern password-hashing function such as Argon2id, bcrypt, or scrypt, with a unique salt. Never store plaintext passwords.

SC-03 — Hard-Coded Secret

Vulnerable pattern: `app.secret_key = 'mysecret123'`.

Impact: Anyone who obtains the source code may be able to forge or compromise signed session data, depending on application configuration.

Remediation: Load secrets from a protected environment/secret store and rotate exposed credentials.

SC-04 — Missing Input Validation

Vulnerable pattern: accepting usernames, emails, and other fields without checking length, format, or allowed characters.

Impact: Invalid or unexpected data can reach application logic and increase attack surface.

Remediation: Validate type, length, format, and allowed values on the server side. Apply allow-list validation where practical.

SC-05 — Cross-Site Scripting (XSS)

Vulnerable pattern: inserting untrusted user content directly into HTML templates without safe escaping.

Impact: Malicious script content could execute in another user's browser if the application renders it unsafely.

Remediation: Use Flask/Jinja auto-escaping and avoid marking untrusted content as safe. Validate input and encode output for its context.

SC-06 — Excessive Error Details

Vulnerable pattern: returning raw database/stack-trace errors to users.

Impact: Internal paths, SQL details, or implementation information may be exposed.

Remediation: Show generic user-facing errors, log detailed diagnostics securely on the server, and disable debug mode in production.

5. Secure Code Examples

Parameterized SQL

```
cursor.execute(
    "SELECT id, password_hash FROM users WHERE username = ?",
    (username,)
)
```

Password hashing

```
from werkzeug.security import generate_password_hash, check_password_hash

password_hash = generate_password_hash(password)
# Later, during login:
if check_password_hash(password_hash, password):
    login_user()
```

Secure secret configuration

```
import os
app.secret_key = os.environ["FLASK_SECRET_KEY"]
```

6. Secure Coding Recommendations

- Treat all external input as untrusted.
- Use parameterized database queries.
- Hash passwords with a password-hashing algorithm designed for credentials.
- Keep secrets out of source code and public repositories.
- Validate input on the server side.
- Use framework output escaping and safe template practices.
- Disable debug mode in production.
- Keep dependencies updated and review known vulnerabilities.
- Use least-privilege database/application accounts.
- Log security events without logging passwords, OTPs, or other secrets.
- Add security-focused tests for authentication, authorization, input validation, and error handling.

7. Remediation Plan

1. Replace all string-built SQL with parameterized queries.
2. Migrate existing plaintext passwords to securely hashed credentials using a controlled reset/migration process.
3. Move application secrets to environment variables or a managed secret store and rotate any exposed keys.
4. Add server-side validation and appropriate length limits.
5. Ensure templates safely escape untrusted content.
6. Replace verbose client-facing errors with generic messages and secure server-side logs.
7. Re-test the application after fixes and add regression tests.

8. Testing Checklist

- SQL queries reviewed for parameterization
- Password database fields checked for secure hashes
- Secrets removed from source code
- Authentication and authorization paths reviewed
- Input validation tested with invalid and boundary values
- User-controlled output tested for safe escaping
- Debug mode disabled for production configuration
- Dependencies checked for known vulnerabilities
- Security fixes retested after remediation

9. Conclusion

The sample application demonstrates several common secure-coding weaknesses that can occur in authentication-focused web applications. The highest-priority issues are SQL injection, plaintext password storage, hard-coded secrets, and unsafe output handling. Applying parameterized queries, secure password hashing, protected secret management, strong validation, and safe output encoding substantially improves the application's security posture.

Prepared for: CodeAlpha Cyber Security Internship — Task 3
Project: Secure Coding Review